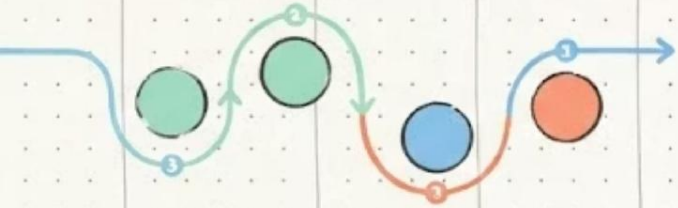


Gestão do Ciclo de Vida da Aplicação

Arquitetura de sistemas (Microsserviços)

Professor Romário da Silva Borges

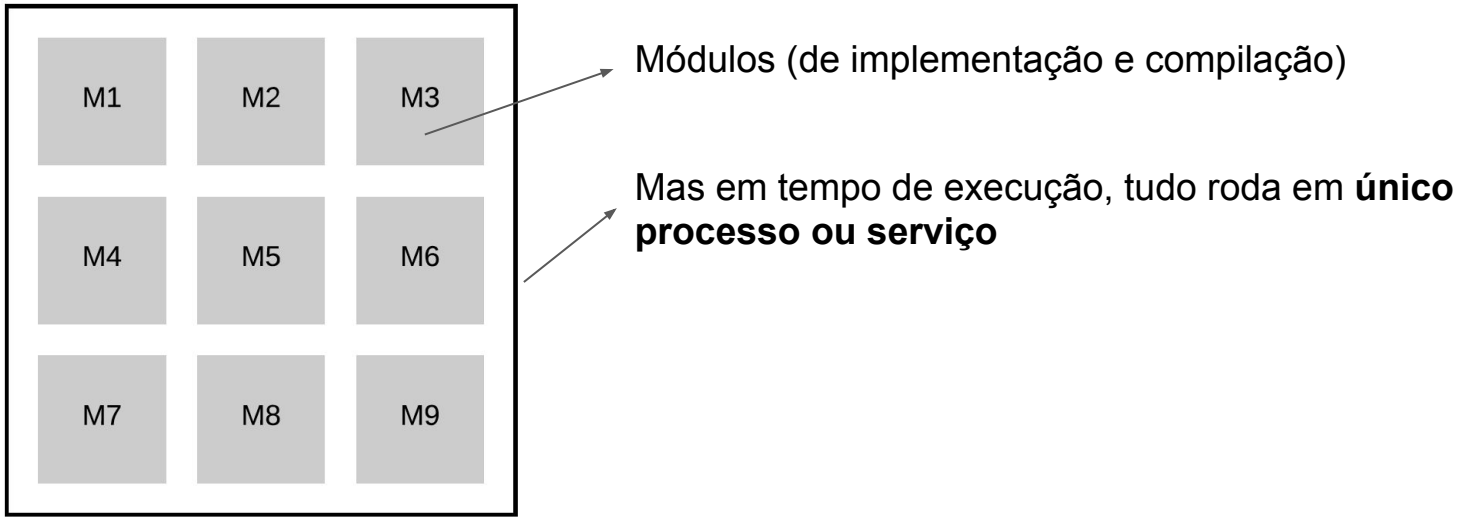


Arquiteturas baseadas em Microsserviços

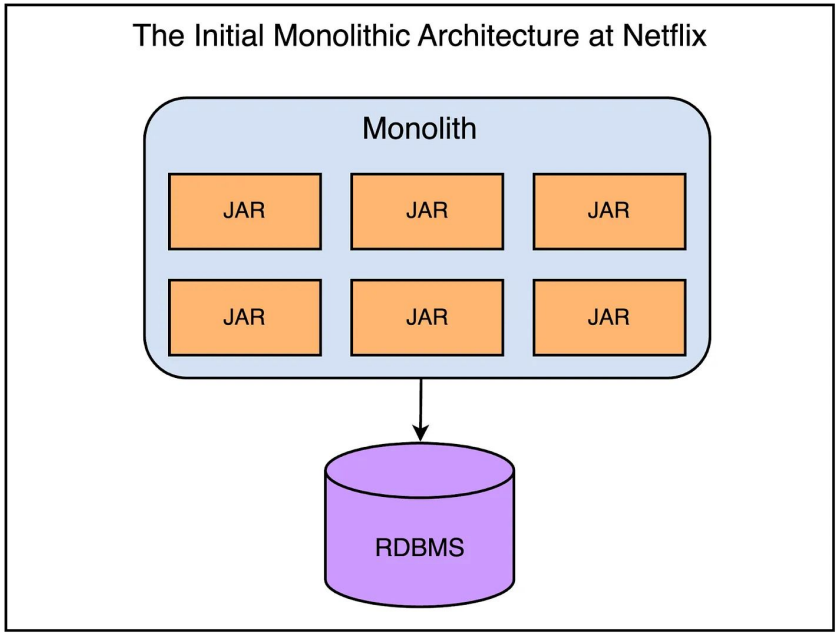
Importante: o que vamos estudar no restante do capítulo diz respeito a arquiteturas de backend

Monolitos

- Em tempo de execução, sistema é um único processo
  (processo aqui é processo de sistema operacional)



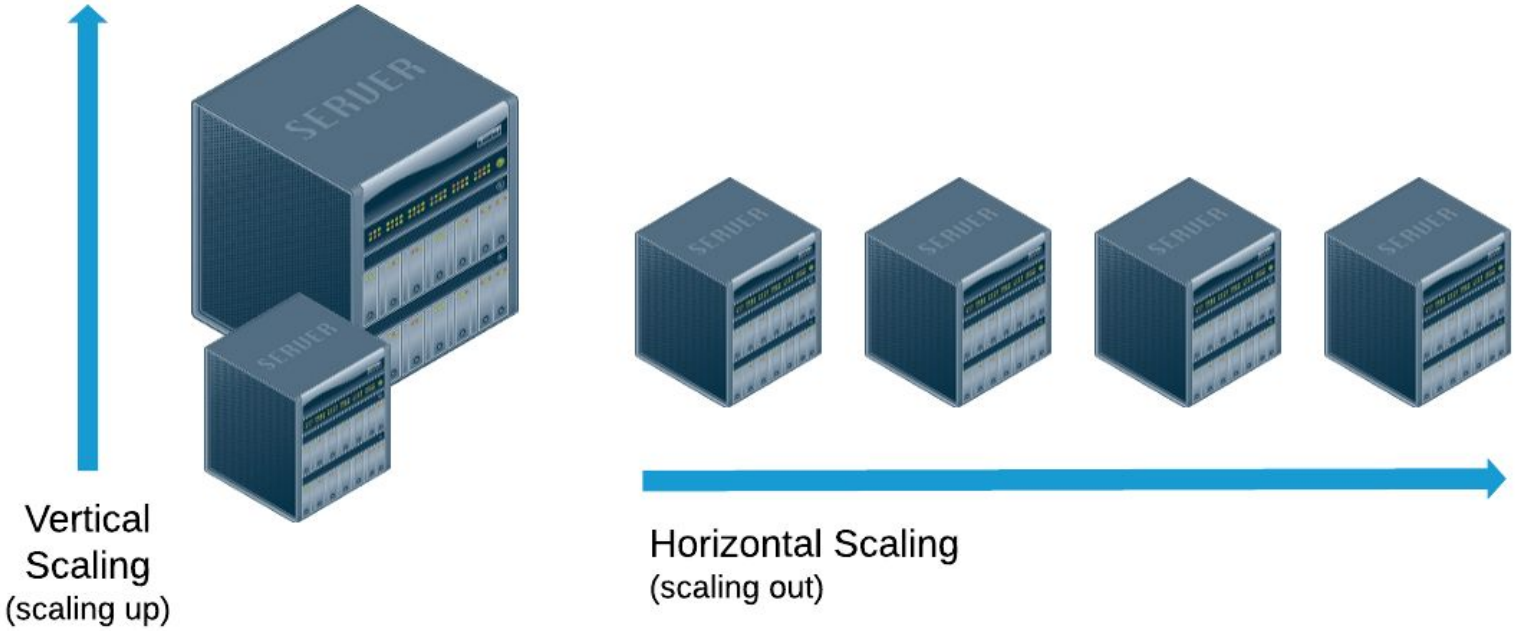
Problema #1: Single Point of Failure



- 2008: grave falha no sistema (BD)
- Dias sem conseguir alugar DVDs
- Duas mudanças importantes:
  - AWS
  - Microsserviços

Fonte: https://blog.bytebytego.com/p/a-brief-history-of-scaling-netflix

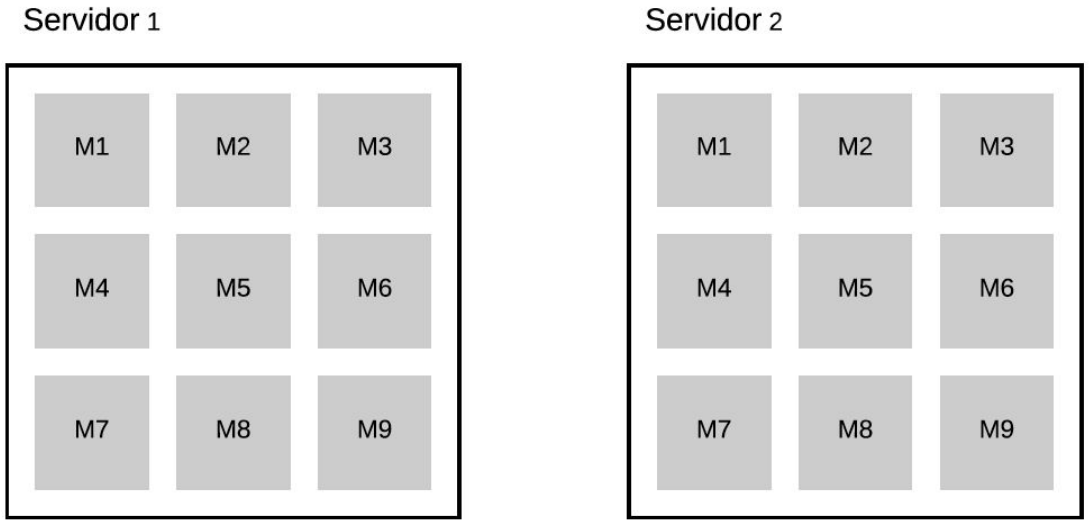
Escalabilidade: Vertical vs Horizontal



https://www.section.io/blog/scaling-horizontally-vs-vertically/

Problema #2 com Monolitos: Escalabilidade

- Escalabilidade horizontal requer escalar o monolito inteiro, mesmo quando o gargalo está em um único módulo



Problema #3: Release é mais lento

- Times não tem poder para colocar módulos em produção
- Motivo: mudanças em um módulo podem impactar módulos que estão funcionando
- Acabam existindo:
  - Datas pré-definidas para release
  - Processo de homologação de releases

Exemplo

- Novo releases: sempre no dia 10
  - Último dia para mudanças: dia 1
  - Testes e homologação: dia dias 2 a 9
- Se implementou uma feature no dia 02/09, mesmo que pequena, ela só entrará em produção no dia 10/10

Riscos de adicionar novas features em um código existente (principalmente, se monolítico)

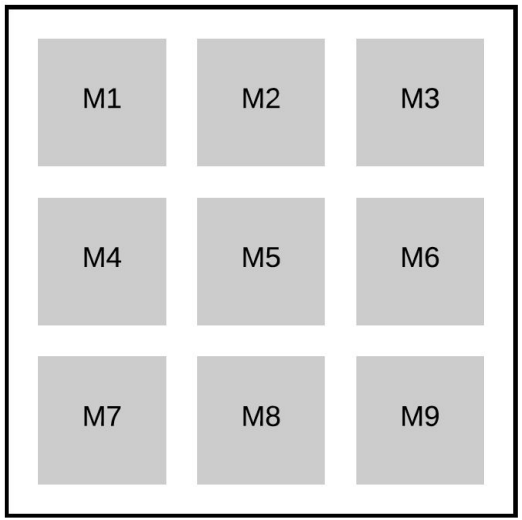


Microsserviços

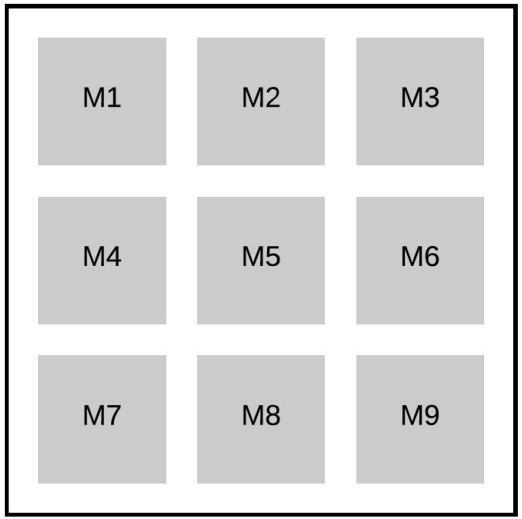
Microsserviços

- Módulos (ou conjuntos de módulos) viram processos em tempo de execução
- Esses módulos são menores do que de um monolito
- Daí o nome **microsserviço**

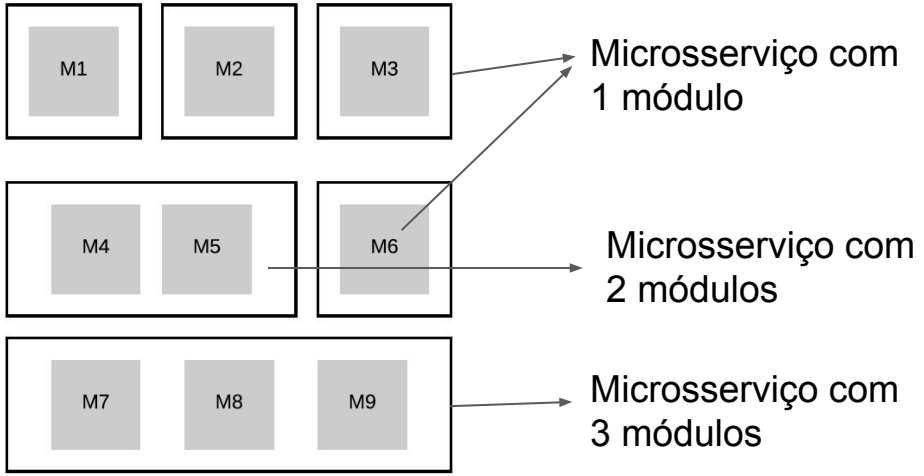
Arquitetura Monolítica



Arquitetura Monolítica



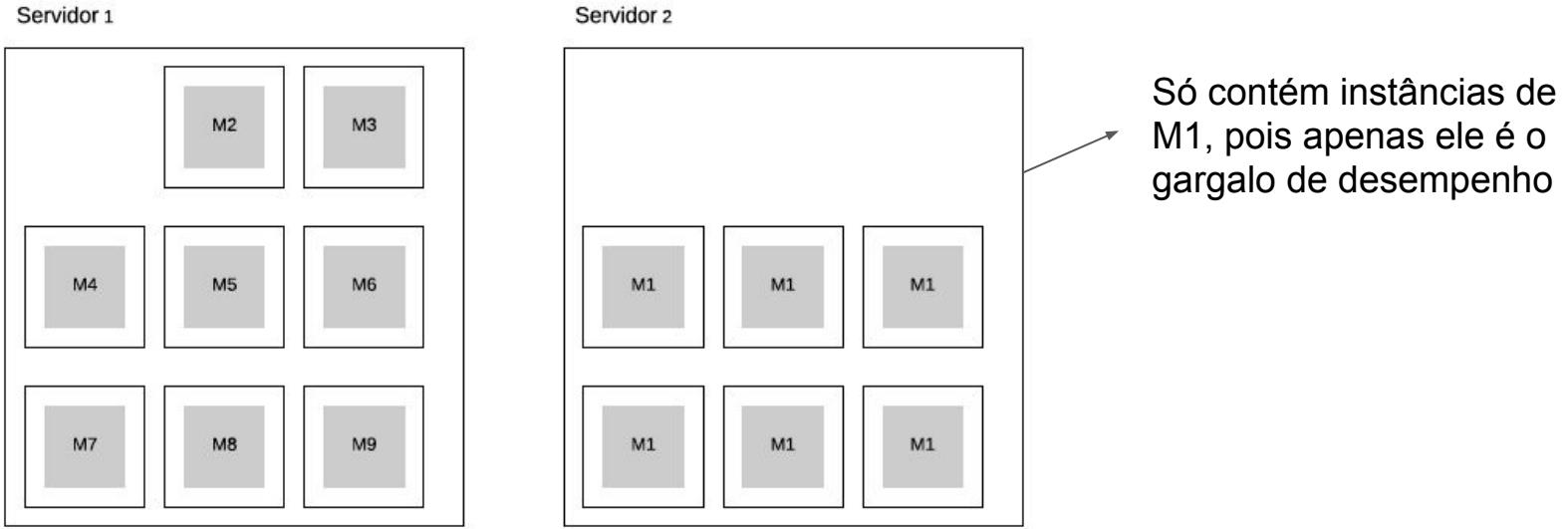
Arquitetura baseada em Microsserviços



microsserviço = processo (run-time, sistema operacional)

Vantagem #1: Escalabilidade

- Pode-se escalar apenas o módulo com problema de desempenho



Vantagem #2: Autonomia de Deployment

- Times ganham autonomia para colocar microsserviços em produção
- Processo = espaço de endereçamento próprio
- Chances de interferências entre processos são menores

Releases de microsserviços

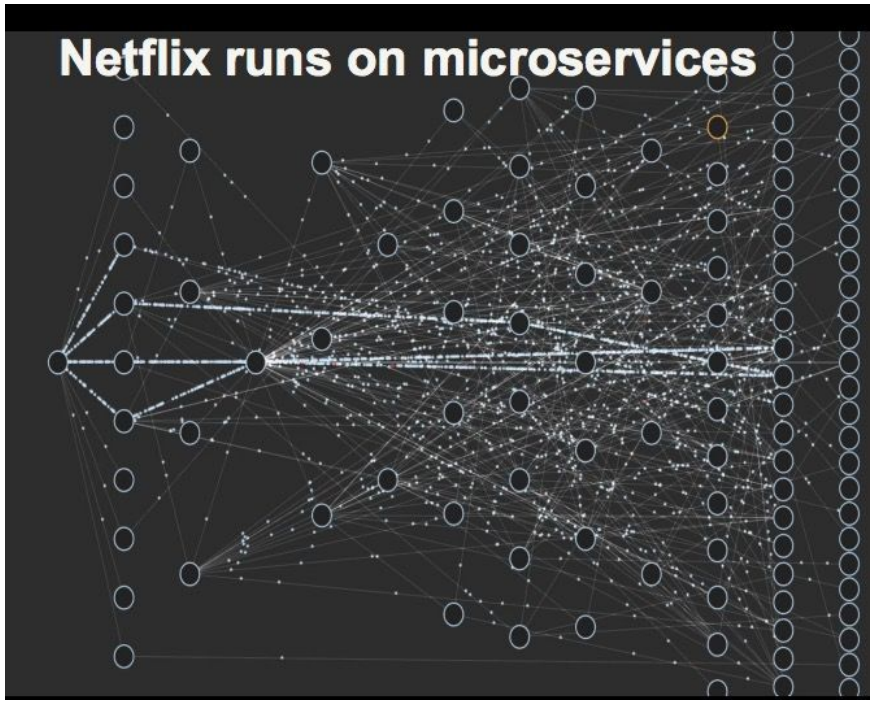
- Não existe mais uma homologação centralizada
- Cada time homologa seus microsserviços
- Se implementou uma feature no dia 02/09, ela pode entrar em produção imediatamente

Outras vantagens

- Tecnologias diferentes
- Falhas parciais

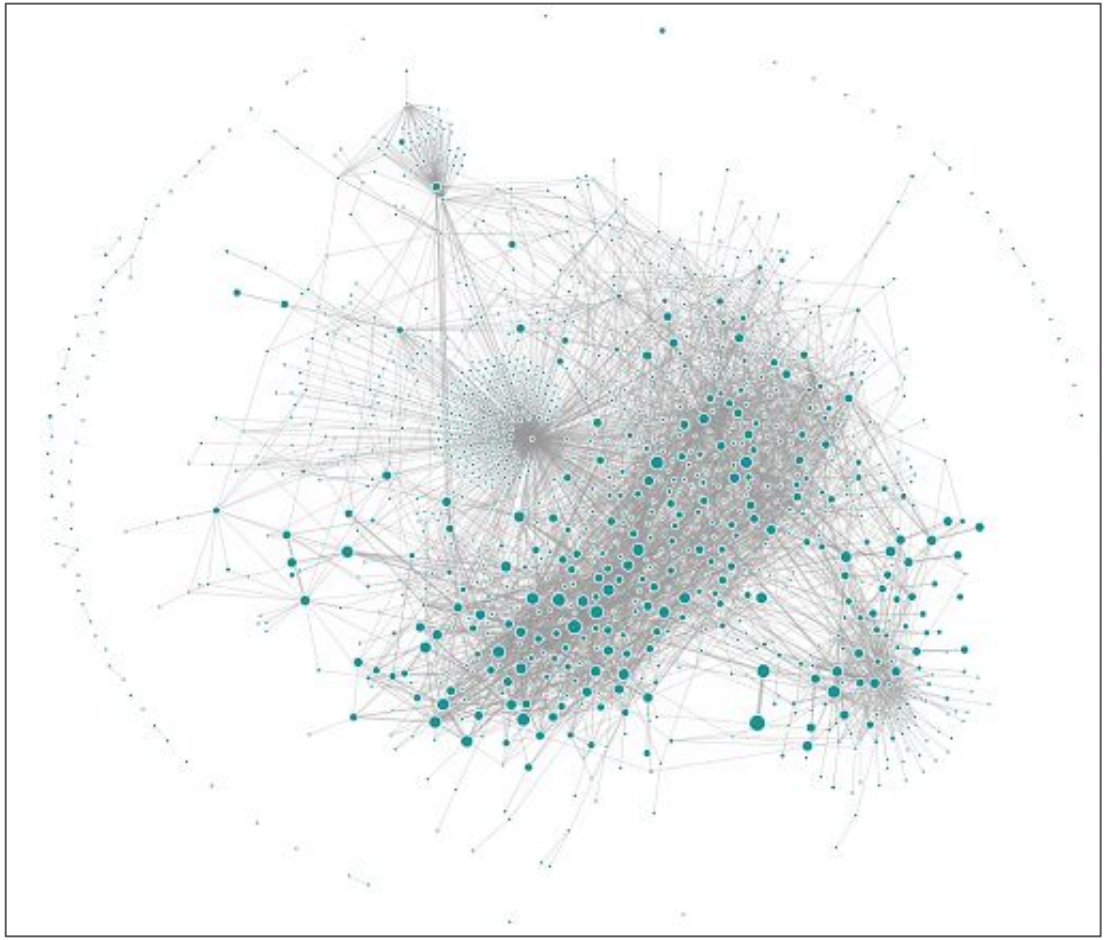
Quem usa microsserviços?

- Grandes empresas como Netflix, Amazon, Google, etc



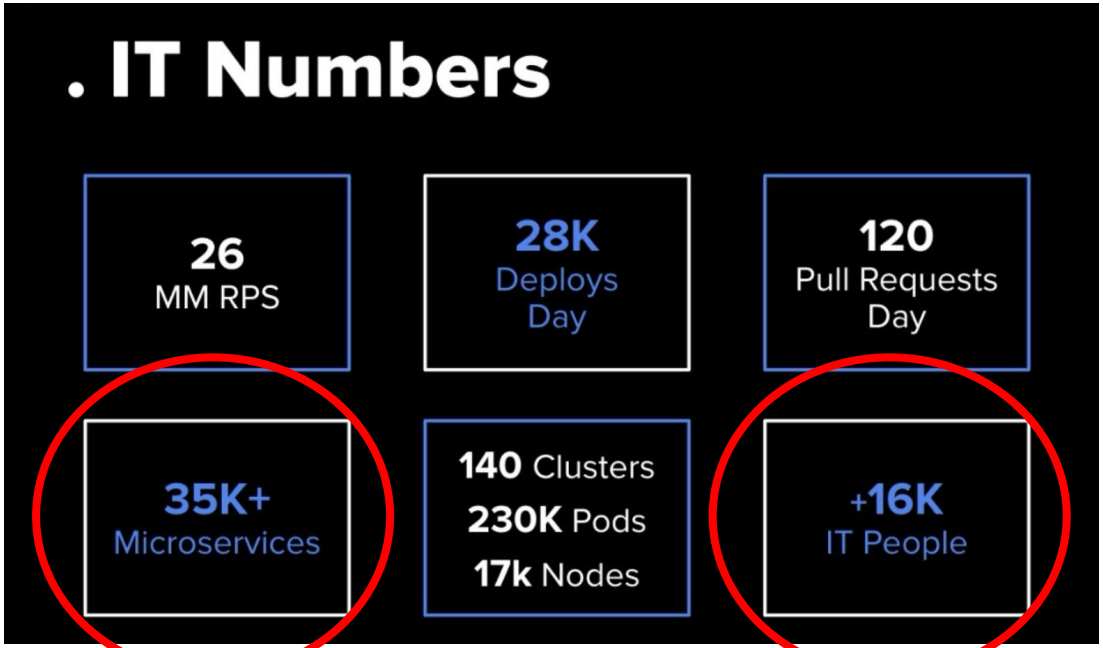
Cada nodo é um microsserviço

Uber (~2018)



https://eng.uber.com/microservice-architecture/

Mercado Livre (maio 2025)



Pergunta: Conhece alguma empresa brasileira que usa microsserviços?

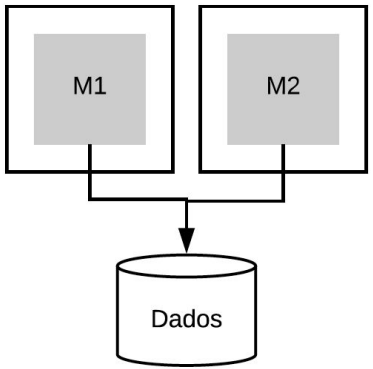
Exemplo de uma aplicação de ecommerce baseada em microsserviços

Team	Framework	Repository
Core	FastAPI	auth-api
Core	FastAPI	fraud-api
Core	FastAPI	group-watcher-api
Core	FastAPI	ledger-composer-api
Core	FastAPI	twilio-api
Core	FastAPI	user-api
Core	FastAPI	whatsapp-instance-api
Core	FastAPI	whatsapp-reader-api
Core	FastAPI	whatsapp-writer-api

Fulfillment	FastAPI	community-api
Fulfillment	FastAPI	hub-api
Fulfillment	FastAPI	invoice-api
Fulfillment	FastAPI	package-api
Fulfillment	FastAPI	refund-api
Fulfillment	FastAPI	sorting-api
Fulfillment	FastAPI	supplier-exports-api
Fulfillment	FastAPI	supplier-payment-api
Fulfillment	FastAPI	support-ticketing-api
Fulfillment	FastAPI	tms-service-api
Fulfillment	FastAPI	wms-service-api

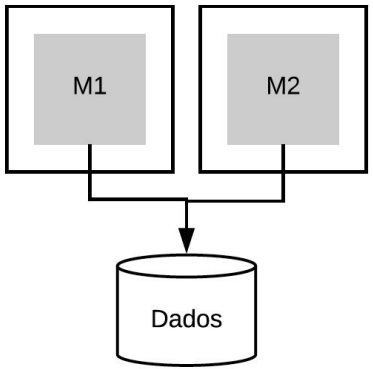
Consumer	FastAPI	coupon-api
Consumer	FastAPI	emporium-composer-api
Consumer	FastAPI	emporium-promo-api
Consumer	FastAPI	events-tagging-api
Consumer	FastAPI	google-tag-manager-api
Consumer	FastAPI	mgm-api
Consumer	FastAPI	orders-api
Consumer	FastAPI	product-recommendation-api
Consumer	FastAPI	product-review-api
Consumer	FastAPI	product-sample-api
Consumer	FastAPI	search-params-api
Consumer	FastAPI	user-lists-api
Consumer	FastAPI	user-notifications-api

Gerenciamento de Dados com Microsserviços

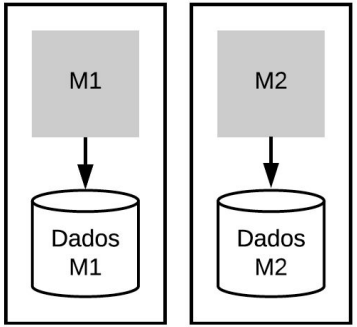


Arquitetura que **não** é recomendada.
Motivo: aumenta acoplamento entre M1 e M2

Gerenciamento de Dados com Microsserviços



❌



✅

Quando não usar microsserviços?

- Arquitetura com microsserviços é mais complexa
  - Sistema distribuído (monitorar centenas de processos)
  - Latência (comunicação via rede)
  - Transações distribuídas
  - Monitoramento e observabilidade

Recomendação: começar com monolitos

- E migrar para microsserviços se:
  - Monolito apresentar problemas de desempenho
  - Monolito estiver atrasando as releases
- Migração pode ser gradativa…

Mas existem exceções



Microsserviços no Nubank, uma visão geral

O que aprendemos ao desenvolver um sistema complexo nos primeiros seis anos de uma fintech

3

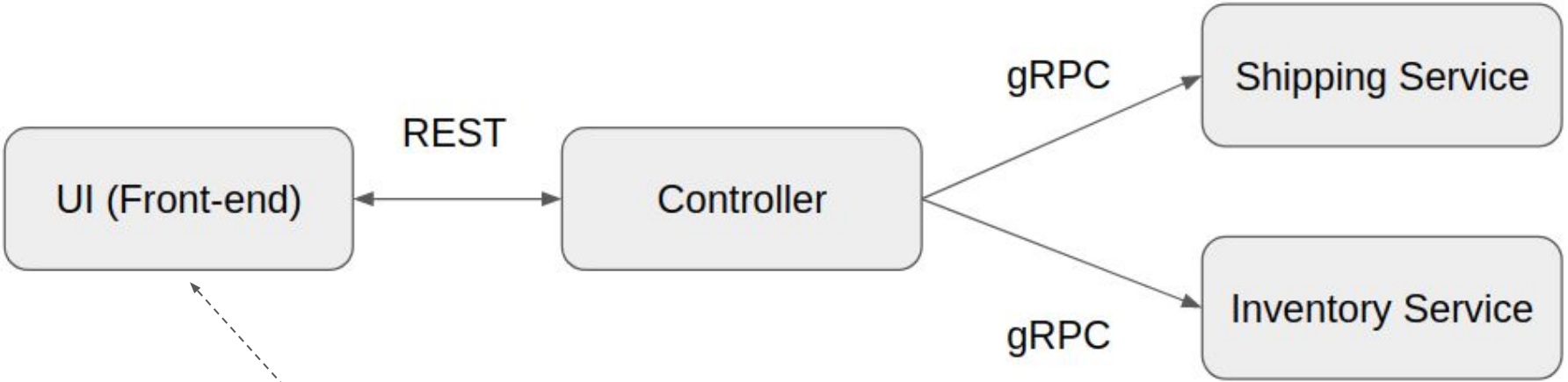
12 Aug, 19 Software Engineering by Rafael Ferreira – Software Engineer

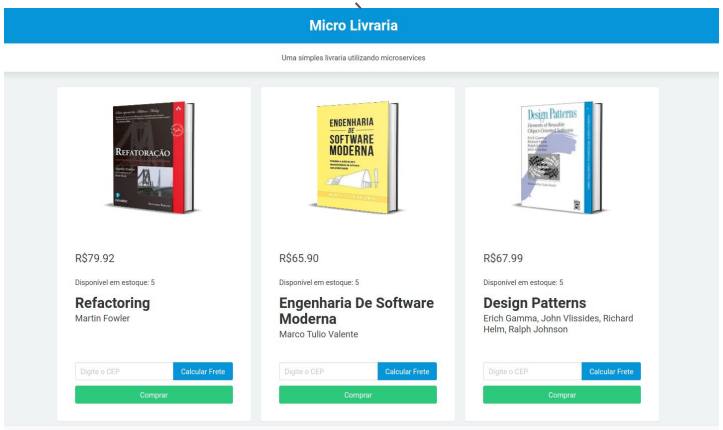
"começamos com microsserviços desde o primeiro dia, desafiando o conselho padrão de começar com um monólito."

https://building.nubank.com/pt-br/microsservicos-no-nubank-uma-visao-geral/

Roteiro prático sobre microsserviços

https://github.com/aserg-ufmg/micro-livraria

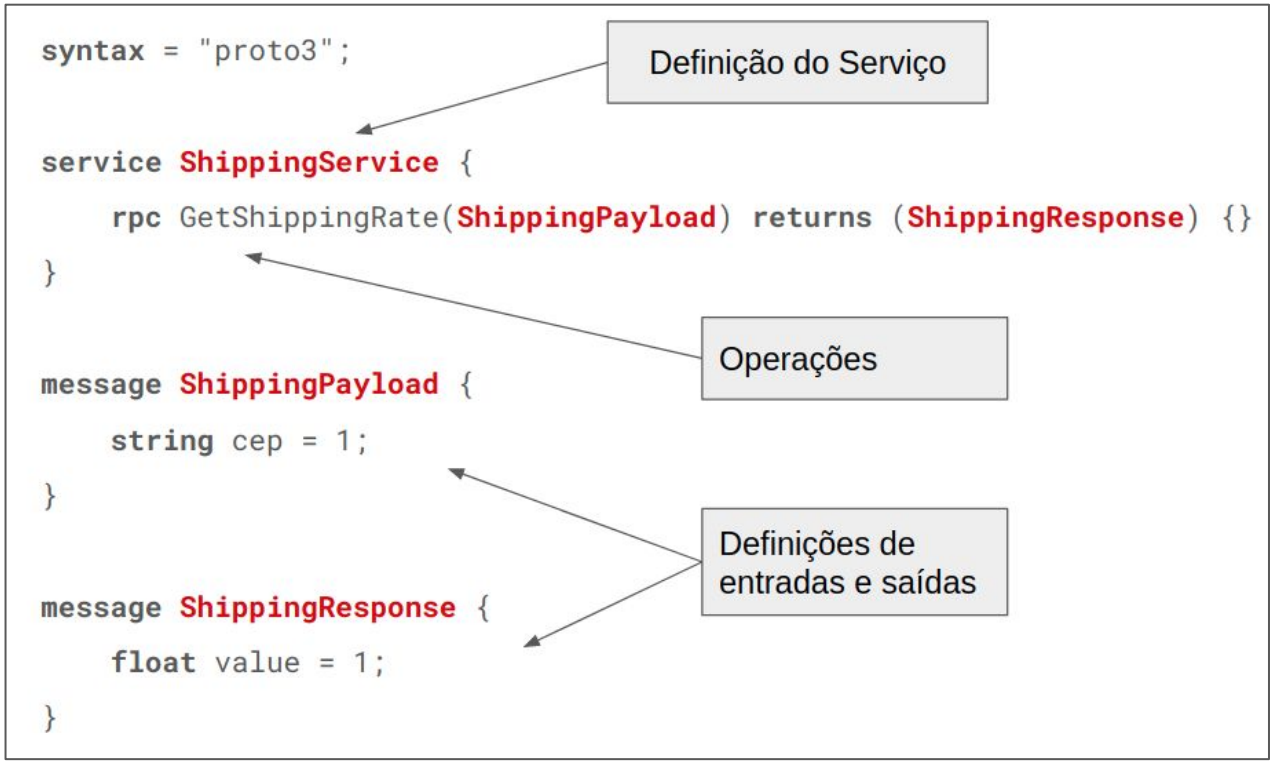

UI (Front-end)
REST
Controller
gRPC
Shipping Service
gRPC
Inventory Service


Micro Livraria
Uma simples livraria utilizando microservices
R$79.92
Disponível em estoque: 5
Refactoring
Martin Fowler
Digite o CEP
Calcular Frete
Comprar
R$65.90
Disponível em estoque: 5
Engenharia De Software Moderna
Marco Tulio Valente
Digite o CEP
Calcular Frete
Comprar
R$67.99
Disponível em estoque: 5
Design Patterns
Erich Gamma, John Vlissides, Richard Helm, Ralph Johnson
Digite o CEP
Calcular Frete
Comprar

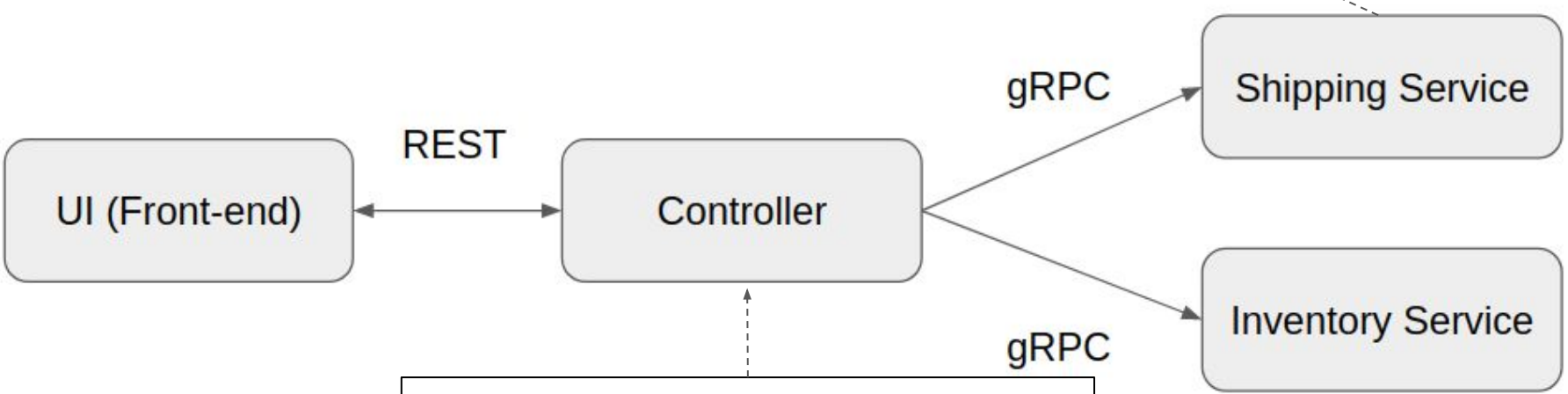
gRPC

- Sistema que viabiliza comunicação entre aplicações
- Baseado em Chamada Remota de Procedimentos
- Inclui um protocolo binário para comunicação distribuída
- E também uma linguagem para definição de interfaces
  - Clientes: chamam métodos dessa interface
  - Servidores: implementam métodos

gRPC: Linguagem para definição de interfaces



```
server.addService(shippingProto.ShippingService.service, {
    GetShippingRate: (_, callback) => {
        const shippingValue = Math.random() * 100 + 1;
        callback(null, {
            value: shippingValue,
        });
    },
});
```




```
app.get('/shipping/:cep', (req,res,next) => {
  shipping.GetShippingRate( {
    cep: req.params.cep,
  },
  (err, data) => { …
```



Professor Marco Tulio Valente

Slides disponibilizados pelo Prof. Marco Tulio Valente. Esta apresentação contém adaptações da apresentação original que está disponível na seguinte página: https://engsoftmoderna.info.

HORSTMANN, C.

Padrões e Projeto Orientados a Objetos, 2 Edição, Editora Bookman: Porto Alegre, 2007

BEZERRA, E.

Princípios de Análise e Projeto de Sistemas com UML, 2a ed., Editora Campus/Elsevier: Rio de Janeiro, 2007

SOMMERVILLE, I.

Engenharia de Software. 8a ed., Ed. Pearson: São Paulo, 2008.

PAULA, Filho W. P.

Engenharia de Software: Fundamentos, Métodos e Padrões, 3a ed. Ed. LTC: Rio de Janeiro, 2009.

PRESSMAN, R.

Software Engineering: A Practitioner's Approach. 5a ed. McGraw Hill : São Paulo, 2000

